

Algorithm 2

Christian MULOMBA

July 2025

1 Introduction

This document presents a synthetic traffic data generation algorithm with causal structure.

Notation

Symbol	Meaning
Q	Road quality
D	Base delay
V	Vehicle counts
F_t	Temporal factor
α	Autocorrelation strength (0.3)
τ	Treatment effect

Key Equations

- Spatial Autocorrelation:

$$V_{xy}^{\text{new}} = (1 - \alpha)V_{xy} + \frac{\alpha}{|\mathcal{N}|} \sum_{(i,j) \in \mathcal{N}(x,y)} V_{ij}$$

- Treatment Effect:

$$\mathbb{E}[Y^1 - Y^0 | Q] = \tau Q^{0.5}$$

Algorithm 1 Enhanced Traffic Data Generation

```

1: Input: GRID_SIZE  $\leftarrow$  10, DAYS  $\leftarrow$  90, FREQ  $\leftarrow$  "15T"  $\triangleright$  Define grid
   size, duration, and time resolution
2: Output: Dataset  $\mathcal{D}$  with causal structure  $\triangleright$  Final dataset includes
   treatment and outcome variables
3: for  $t = 1$  to  $4 \times 24 \times \text{DAYS}$  do  $\triangleright$  Loop over all time intervals (15-min
   steps for 90 days)
4:   for each  $(x, y)$  in  $1 : \text{GRID\_SIZE}$  do  $\triangleright$  Loop over each zone in the grid
5:      $k \leftarrow \text{zone\_type}(x, y)$   $\triangleright$  Assign zone type: residential, commercial, or
     mixed
6:      $Q_{xy} \sim \mathcal{U}(0.5, 2.0)$   $\triangleright$  Sample road quality uniformly
7:      $D_{xy} \sim \mathcal{N}(\mu_k, \sigma_k^2)$   $\triangleright$  Sample base delay based on zone type
8:      $V_{xy} \sim \text{Poisson}(\lambda_k)$   $\triangleright$  Sample vehicle count from Poisson distribution
9:   end for
10:   $V_t \leftarrow \text{calculate\_vehicle\_counts}(V, Q, F_t)$   $\triangleright$  Compute traffic flow using
   road quality and temporal factors
11:   $V_t \leftarrow (1 - \alpha)V_t + \alpha\mathcal{N}(V_t)$   $\triangleright$  Apply autocorrelation to smooth traffic
   counts
12:   $P_t \leftarrow 0.2 + 0.35H_t - 0.15W_t + 0.05C + 0.05(Q - 1.0)$   $\triangleright$  Compute
   treatment probability based on time, zone, and quality
13:   $T_t \sim \text{Bernoulli}(\max(0.1, \min(0.7, P_t)))$   $\triangleright$  Sample toll activation from
   clipped probability
14:   $Y_t^0 \leftarrow D \odot (1.2 - 0.1Q) \odot \epsilon_t$   $\triangleright$  Potential outcome without treatment
   (baseline delay with noise)
15:   $Y_t^1 \leftarrow Y_t^0 \odot (1 - \tau T_t)$   $\triangleright$  Apply treatment effect to delay
16:   $Y_t \leftarrow Y_t^1 + R_t + E_t$   $\triangleright$  Add rain and event effects to get final delay
17: end for
18:  $\text{ATE} \leftarrow \text{mean}(Y|T = 1) - \text{mean}(Y|T = 0)$   $\triangleright$  Estimate average treatment
   effect
19: Return:  $\mathcal{D} \leftarrow \{t, V, Y, T, Q, H, W\}$   $\triangleright$  Return full dataset with all relevant
   variables

```

Algorithm 2 Preprocess Data for Causal Modeling

- 1: **Input:** Raw dataset df , list of lags $[1, 3, 6]$
 - 2: **Output:** Features X , outcome Y , treatment T , fitted preprocessor, cleaned dataframe
 - 3: $df \leftarrow \text{copy}(df)$ ▷ Avoid modifying original data
 - 4: Convert `zone_type` to categorical ▷ Prepare for dummy encoding
 - 5: Convert `is_peak`, `is_weekend` to binary integers ▷ Ensure consistent format
 - 6: Extract spatial coordinates (x, y) from `zone_id` ▷ Enable spatial modeling
 - 7: **for** each lag in $[1, 3, 6]$ **do**
 - 8: Create lagged features for `final_delay`, `vehicle_count`, `base_delay` ▷ Capture temporal dependencies
 - 9: **end for**
 - 10: Drop rows with missing values in essential columns ▷ Ensure complete cases for modeling
 - 11: Create one-hot encoded dummies for `zone_type` ▷ Enable zone-type interactions
 - 12: **for** each dummy column **do**
 - 13: Create interactions with `is_peak`, `is_weekend`, `rain` ▷ Capture contextual heterogeneity
 - 14: **end for**
 - 15: Create interactions between `road_quality` and `is_peak`, `is_weekend` ▷ Model quality-time effects
 - 16: Define confounders: original features, lags, dummies, and interactions ▷ Comprehensive feature set for causal inference
 - 17: Define preprocessing pipeline:
 - 18: Spline + scaling for numeric features
 - 19: Scaling for interaction features
 - 20: Pass-through for binary and zone dummies
 - 21: Fit and transform $X \leftarrow \text{preprocessor.fit_transform}(df[\text{confounders}])$
 - 22: Extract outcome $Y \leftarrow df[\text{final_delay}]$
 - 23: Extract treatment $T \leftarrow df[\text{toll_active}]$
 - 24: **Return:** X, Y, T , preprocessor, df
-

2 Preprocessing Pipeline

Algorithm 3 Economic Toll Activation Policies

```

1: Global Parameters:
2: value_per_minute  $\leftarrow$  0.10 ▷ USD per minute of reduced delay
3: toll_cost  $\leftarrow$  0.08 ▷ USD per toll activation
4: max_activations  $\leftarrow$  15000 ▷ Budget constraint
5:
6: procedure FIXEDTHRESHOLDPOLICY(df_sampled)
7:   fixed_threshold  $\leftarrow$  quantile(df_sampled['cate'], 0.25)
8:   df_fixed  $\leftarrow$  copy(df_sampled)
9:   df_fixed['sim_toll']  $\leftarrow$  (df_fixed['cate']  $\leq$  fixed_threshold)
10:  df_fixed['sim_delay']  $\leftarrow$  df_fixed['final_delay'] + df_fixed['cate']  $\times$ 
    df_fixed['sim_toll']
11:  return df_fixed
12: end procedure
13:
14: procedure DYNAMICTHRESHOLDPOLICY(df_sampled)
15:   df_dynamic  $\leftarrow$  copy(df_sampled)
16:   df_dynamic['expected_savings']  $\leftarrow$  |df_dynamic['cate']|  $\times$ 
    value_per_minute
17:   df_dynamic['sim_toll']  $\leftarrow$  (df_dynamic['expected_savings']  $>$  0.15)
18:   if sum(df_dynamic['sim_toll'])  $>$  max_activations then
19:     threshold  $\leftarrow$  min(nlargest(df_dynamic['expected_savings'], max_activations))
20:     df_dynamic['sim_toll']  $\leftarrow$  (df_dynamic['expected_savings']  $\geq$ 
    threshold)
21:   end if
22:   df_dynamic['sim_delay']  $\leftarrow$  df_dynamic['final_delay'] +
    df_dynamic['cate']  $\times$  df_dynamic['sim_toll']
23:   return df_dynamic
24: end procedure
25:
26: procedure HYBRIDPOLICY(df_sampled, dynamic_thresh)
27:   df_hybrid  $\leftarrow$  copy(df_sampled)
28:   df_hybrid['expected_savings']  $\leftarrow$  |df_hybrid['cate']|  $\times$  value_per_minute
29:   df_hybrid['sim_toll']  $\leftarrow$  (df_hybrid['expected_savings']  $>$ 
    dynamic_thresh)
30:   remaining  $\leftarrow$  max_activations - sum(df_hybrid['sim_toll'])
31:   if remaining  $>$  0 then
32:     candidates  $\leftarrow$  nsmallest(df_hybrid[df_hybrid['sim_toll']] ==
    0], remaining, 'cate')
33:     df_hybrid.loc[candidates.index, 'sim_toll']  $\leftarrow$  1
34:   end if
35:   df_hybrid['sim_delay']  $\leftarrow$  df_hybrid['final_delay'] + df_hybrid['cate']  $\times$ 
    df_hybrid['sim_toll']
36:   return df_hybrid
37: end procedure
38:
39: procedure SUMMARIZEPOLICY(df, label)
40:   savings  $\leftarrow$  sum(df['final_delay'] - df['sim_delay'])
41:   cost  $\leftarrow$  sum(df['sim_toll'])  $\times$  toll_cost
42:   return {Policy : label, Delay Reduction : savings/60, ...}
43: end procedure

```

Algorithm 4 Quota-Hybrid Toll Policy

Example quotas: $\begin{cases} \text{residential} = 0.25, \\ \text{mixed} = 0.15, \\ \text{commercial} = 0.60 \end{cases}$